

برنامه نویسی پیشرفته

زمستان ۱۴۰۲ و بهار ۱۴۰۳ - دانشکده علوم ریاضی
دانشگاه صنعتی شریف

تمرین سری سوم

زمان تحویل: ۱۴ اردیبهشت

مقدمه و اهداف تمرین

در این تمرین قصد داریم یکی از کلاس های معروف جاوا به نام MyMap را پیاده‌سازی کنیم تا با بیشتر با مفاهیم Generic و Wildcard آشنا شویم. به طور کلی از شما انتظار می رود که یک کلاس به نام MyMap ایجاد کنید که تمام ویژگی های خواسته شده زیر را داشته باشد.

- توجه داشته باشید که در این تمرین استفاده از هرگونه از کلاس های آماده جاوا مانند: List, ArrayList, Map و ... مجاز نمی باشد و تنها می‌توانید از آرایه استفاده کنید.
- در همه‌ی بخش‌های این تمرین باید تا جای ممکن از مفاهیم جنریک و به خصوص وایلد کارت استفاده کنید.
- استفاده از ابزارهای جنریت کردن کد **ممنوع** می‌باشد.

طراحی کلاس مپ

این کلاس یک کلاس Generic است که دو تایپ از کاربر میگیرد و از آن ها برای تایپ های key و value به ترتیب استفاده میکند (مشابه Map) شما باید حتما توابع زیر را در این کلاس پیاده سازی کنید.

get: این تابع از شما یک کلید به عنوان ورودی میگیرد و در صورت وجود آن کلید در کلاس شما مقدار آن را بر میگرداند. توجه داشته باشید که اگر کلید وجود نداشت مقدار null را باز گردانید.

getOrDefault: این تابع مشابه تابع get عمل میکند با این تفاوت که در ورودی های خود علاوه بر کلید یک مقدار اولیه نیز میگیرد و در صورت عدم وجود کلید، مقدار اولیه را باز میگرداند.

put: این تابع یک کلید و مقدار میگیرد و آن ها را به مپ اضافه میکند. Value اضافه شده را ریترن می‌کند. در صورت تکراری بودن کلید باید مقدار آن را آپدیت کنید.

indexOf: اندیس مربوط به یک کلید را برمی‌گرداند.

putIfAbsent: همانند تابع put عمل میکند با این تفاوت که تنها در صورتی مپ را آپدیت میکند که کلید وارد شده در مپ وجود نداشته باشد.

keys: یک آرایه از کلید های موجود در مپ را برمیگرداند.

values: یک آرایه از مقدار های موجود در مپ را برمیگرداند.

putAll: یک آبجکت از جنس MyMap را میگیرد و آن را به مپ اضافه میکند. (توجه داشته باشید که برای پیاده سازی این بخش الزامی است که از Wildcard استفاده کنید)

containsKey: یک کلید را گرفته و وجود آن را در مپ بررسی می‌کند.

containsValue: یک مقدار را گرفته و وجود آن را در مپ بررسی می‌کند.

size: سایز مپ را بر میگرداند.

clear: تمام عناصر درون مپ را پاک میکند.

remove: یک کلید را گرفته و آن را از مپ حذف می‌کند و value کلید حذف شده را ریترن می‌کند.

remove: یک اندیس را گرفته و آن را از مپ حذف می‌کند و value کلید حذف شده را ریترن می‌کند (آرگومان متفاوتی را می‌پذیرد).

removeAll: یک آرایه از کلید ها میگیرد و آن ها را از مپ حذف می‌کند.

removeIfValue: یک تابع که شیای از نوع Predicate ورودی می‌گیرد که شرطیست روی valueهای مپ. در صورت درست بودن شرط برای هر valueای جفت مربوط به آن value از مپ حذف می‌شود.

removeIfKey: یک تابع که شیای از نوع Predicate ورودی می‌گیرد که شرطیست روی keyهای مپ. در صورت درست بودن شرط برای هر keyای جفت مربوط به آن key از مپ حذف می‌شود.

sort: مرتب کردن جفت‌های درون مپ با ورودی گرفتن یک شی از نوع Comparator که شرطی است فقط بر روی کلیدهای مپ.

iterator: یک متد که شی خروجی آن از نوع اینترفیس MyIterator است را بر می‌گرداند. این اینترفیس در پروژه اولیه قرار دارد و شما باید متدهای خواسته شده را override کنید. تمامی متدها برای اینترفیس ListIterator هستند و دو متد جدید داریم. nextByAmount و

prevByAmount. این دو درست مانند متدهای previous و next عمل می‌کنند با این تفاوت که به جای یک المان به تعداد ورودی به عقب یا جلو می‌روند.

replaceAll: یه شی از جنس BiFunction می‌پذیرد و تمامی value ها یا بعضی از آن‌ها را با توجه به ورودی تغییر می‌دهد.

compute: یک key و یک شی از جنس BiFunction می‌پذیرد و با توجه به شی BiFunction مقدار key مورد نظر را آپدیت می‌کند و value جدید را ریترن می کند.

نکات تکمیلی:

1. برای درک بهتر بعضی توابع می‌توانید به کلاس HashMap مراجعه کنید.
2. اینترفیس MyListIterator در پروژه اولیه قابل دانلود در صفحه سوال کوئرا قرار دارد.